CÓDIGOCLARO · MATERIAL DE ESTUDIO

# Java y programación orientada a objetos

Cuaderno DAM/DAW · Java SE 21 · API estándar, sin funciones preview.

## Del código fuente a la JVM

Objetivo: Explicar qué comprueba javac y qué ejecuta la JVM al arrancar un proyecto.

Java comprueba sintaxis y tipos al compilar; normalmente produce bytecode que la JVM carga y ejecuta. Separar compilación y ejecución ayuda a localizar un error: un símbolo desconocido se corrige antes de ejecutar, mientras que una clase no encontrada apunta a ruta o classpath. El nombre del archivo debe corresponder a la clase pública. El ejemplo evita funciones preview y puede compilarse con Java SE 21; anota la versión usada para reproducir el resultado.

Código · Java

```
public class Main {
  public static void main(String[] args) {
    System.out.println("DAM listo");
  }
}
// javac Main.java
// java Main
```

Lectura del ejemplo: javac analiza el archivo indicado y genera una clase compilada; java carga esa clase y busca main como entrada del programa. Si se declara package centro, el archivo va en la ruta centro y se ejecuta usando el nombre cualificado desde la raíz del classpath. Para diagnosticar “class not found”, anota la carpeta actual y la ruta de clases. El bytecode separa el código fuente de la ejecución, pero las clases dependientes también deben estar disponibles.

Práctica: Compila Main.java y ejecútalo desde otra carpeta. ¿Qué debe estar disponible para que Java encuentre Main y cómo verificarías el directorio de trabajo?

Respuesta razonada: javac crea Main.class en el directorio actual; java necesita un classpath que incluya ese directorio, o la raíz adecuada si la clase declara paquete. Verifica la carpeta, la ruta del archivo y el comando de ejecución. Un error de compilación no se corrige cambiando la ejecución y viceversa.

Documentación consultada: Java Language Specification, Java SE 21

# Tipos y decisiones

Objetivo: Elegir un tipo adecuado y expresar explícitamente las condiciones de una matrícula.

Java es estáticamente tipado: una condición debe producir boolean y los números no se convierten automáticamente en verdadero o falso. if/else resuelve una decisión; switch expresa alternativas discretas y for recorre una secuencia conocida. String es una referencia: == compara identidad y equals compara contenido. Usa enteros para contadores; si el enunciado exige importes exactos, no supongas que double representa cada decimal con exactitud. Prueba el límite y un valor a cada lado.

Código · Java

```
int plazas = 0;
if (plazas < 0) throw new IllegalArgumentException();
String estado = plazas > 0 ? "abierta" : "completa";
for (int i = 0; i < plazas; i++) System.out.println(i + 1);
String a = new String("DAM");
System.out.println(a.equals("DAM"));
```

La condición se decide con un boolean; por eso una comparación como plazas > 0 produce una decisión explícita. switch resulta útil cuando se comparan estados discretos y for cuando el número de vueltas se conoce. La validación inicial evita que un dato fuera del dominio se confunda con “completa”. Para cadenas no compares con == si preguntas por el texto: usa equals y decide qué hacer si la referencia pudiera ser null.

Práctica: Prueba plazas 0, 1 y -1; luego compara a == "DAM" con a.equals("DAM"). ¿Qué regla expresa cada comparación?

Respuesta razonada: -1 se rechaza por la validación; 0 deja el estado completo y 1 abre una plaza. El bucle recorre una vuelta por plaza. == puede ser falso porque compara referencias distintas; equals devuelve verdadero al comparar el texto. Valida el dominio por separado de la decisión de negocio.

Documentación consultada: Java Language Specification, Java SE 21 · API Java SE 21: String.equals

# Métodos y sobrecarga

Objetivo: Diseñar una operación con contrato verificable y distinguir sobrecarga de retorno.

La firma de un método depende del nombre y los parámetros, no solo del tipo de retorno. Un método debe expresar una responsabilidad, validar entradas y devolver un resultado comprensible. Los argumentos se pasan por valor; en un parámetro de referencia se copia la referencia, por lo que el objeto compartido aún puede observar cambios.

Código · Java

```
static double media(int total, int recuento) {
  if (recuento <= 0) throw new IllegalArgumentException();
  return (double) total / recuento;
}
```

La firma diferencia métodos por nombre y parámetros; cambiar solo el tipo de retorno no crea una sobrecarga válida. El cast a double antes de dividir conserva la parte fraccionaria, mientras que una división entre dos int trunca el resultado. Al añadir la versión para int[], conserva un único contrato para “sin datos”: rechazarlo con excepción o representarlo de manera explícita. Las pruebas deben comprobar tanto la media normal como el caso vacío. En una sobrecarga, el compilador elige la variante según los tipos y cantidad de argumentos; el tipo devuelto se conoce después de esa elección. Para una API pública, documenta qué excepción representa cada entrada inválida.

Práctica: Añade una sobrecarga que acepte un array y define su comportamiento si está vacío.

> Respuesta razonada: La sobrecarga acepta int[] y suma sus elementos. Para el array vacío, rechaza la entrada o devuelve un resultado explícito; dividir entre cero no es una política válida implícita.

Documentación consultada: Java Language Specification, Java SE 21

# Arrays y límites

Objetivo: Recorrer notas sin acceder fuera de los límites ni perder el caso de array vacío.

Los arrays tienen longitud fija y los índices comienzan en cero. Para n elementos, el último índice es n-1 y el bucle habitual usa i < length. El for-each reduce errores si no necesitas la posición. Comprueba la longitud antes de calcular una media: un array vacío no tiene denominador válido.

Código · Java

```
int[] notas = {7, 8, 6};
if (notas.length == 0) {
  throw new IllegalArgumentException("No hay notas");
}
int suma = 0;
for (int nota : notas) suma += nota;
double media = (double) suma / notas.length;
```

El primer índice válido es 0 y el último es length - 1; por eso el límite de un for es i < notas.length. Un for-each evita errores de posición si no se necesita el índice. La suma es entera, así que convierte al calcular la media para no truncar. Prueba el array vacío antes de dividir y uno de longitud uno para verificar que el denominador y el recorrido coinciden. El cast antes de dividir evita que una media como 7.5 se convierta en 7 por división entera. El array conserva su longitud al recorrerlo; no se puede añadir una cuarta nota sin crear otro array.

Práctica: ¿Qué ocurre si sustituyes i < notas.length por i <= notas.length?

Respuesta razonada: El bucle intenta leer el índice length, fuera del array, y lanza ArrayIndexOutOfBoundsException. La media necesita además tratar length igual a cero.

Documentación consultada: Java Language Specification, Java SE 21

# Clases e invariantes

Objetivo: Evitar que una cuenta de prácticas tenga un saldo inicial inválido.

Una clase agrupa estado y operaciones. Los campos privados impiden modificaciones arbitrarias desde fuera; el constructor y los métodos deben preservar las invariantes. Si el saldo no puede ser negativo, valida al construir y antes de retirar. Si una operación se rechaza, el estado debe conservarse sin cambios.

Código · Java

```
final class Cuenta {
  private int saldo;
  Cuenta(int inicial) {
    if (inicial < 0) throw new IllegalArgumentException();
    saldo = inicial;
  }
  int saldo() { return saldo; }
}
```

La regla saldo >= 0 es una invariante: debe ser cierta después del constructor y cada operación. El retiro comprueba el importe antes de restar; si falla, no modifica el estado. Encapsular saldo permite que Cuenta sea la única vía para cambiarlo. El método saldo() permite consultar sin exponer el campo. Evita ofrecer otra ruta que pueda crear o dejar una cuenta fuera del dominio.

La regla saldo >= 0 es una invariante: debe ser cierta después de construir el objeto y tras cada operación. Un retiro se valida antes de restar; si se rechaza, el saldo no cambia. Encapsular el campo permite que Cuenta controle esa regla en un solo lugar. Un método lector puede exponer el valor sin entregar una forma de modificarlo directamente desde código externo. Una cuenta válida encapsula saldo y operaciones. La comprobación ocurre antes de la resta para que un rechazo no deje saldo negativo; un getter no debe permitir cambiar el campo desde otra clase.

Práctica: Define qué debe ocurrir al retirar más dinero del saldo.

> Respuesta razonada: La operación debe rechazar el retiro antes de modificar saldo. Así el objeto no atraviesa un estado inválido y el llamador distingue una operación imposible de una retirada correcta.

Documentación consultada: Dev.java: programación orientada a objetos

# Constructores y this

Objetivo: Usar constructores para crear productos válidos y delegar la inicialización sin duplicarla.

Un constructor puede delegar en otro de su clase mediante this(...), que debe ser la primera instrucción. Los campos final se asignan durante la inicialización y no se pueden volver a reasignar después. Validar nombre y cantidad al crear el objeto evita que otros métodos reciban instancias parcialmente válidas. El constructor corto reutiliza la misma validación al llamar al completo; no dupliques reglas en cada variante.

Código · Java

```
final class Producto {
  final String nombre;
  final int unidades;
  Producto(String nombre) { this(nombre, 0); }
  Producto(String nombre, int unidades) {
    if (nombre.isBlank() || unidades < 0) throw new IllegalArgumentException();
    this.nombre = nombre; this.unidades = unidades;
  }
}
```

La llamada this(nombre, 0) debe ocupar la primera instrucción del constructor delegante, de modo que toda creación use el mismo camino de validación. Los campos final expresan que el valor queda fijado tras construir el objeto. isBlank comprueba texto vacío o formado solo por espacios; si el contrato permite esos nombres, la condición debe cambiarse deliberadamente. El tipo de excepción debe señalar que el argumento incumple el contrato.

Práctica: Define si debe poder crearse Producto con nombre vacío o unidades negativas. ¿Qué constructor recibe primero el control y cuándo se asignan los campos?

Respuesta razonada: No se acepta ninguno de esos valores. El constructor de un argumento delega inmediatamente con this(nombre, 0); después el constructor completo valida y asigna. this.nombre identifica el campo frente al parámetro homónimo. Si la validación falla, no se devuelve una instancia con estado incompleto.

Documentación consultada: Java Language Specification, Java SE 21 · Dev.java: programación orientada a objetos

# Herencia y composición

Objetivo: Decidir si una regla variable debe heredar de una clase o delegarse a un colaborador.

Herencia expresa una relación de sustitución: el subtipo debe mantener las garantías de la base. Composición delega comportamiento y suele permitir intercambiar políticas sin ampliar la jerarquía. No heredes solo para reutilizar unas líneas. El diseño más claro es el que conserva el contrato y limita dependencias.

Código · Java

```
interface Descuento { int aplicar(int precio); }
final class PrecioFinal {
  private final Descuento descuento;
  PrecioFinal(Descuento descuento) { this.descuento = descuento; }
  int calcular(int base) { return descuento.aplicar(base); }
}
```

La composición hace visible la dependencia: PrecioFinal recibe una política Descuento y la utiliza al calcular. La política puede ser fija, promocional o de prueba sin cambiar el consumidor. Con herencia, una subclase debe respetar lo que promete la clase base; si restringe sus operaciones o cambia resultados esperados, deja de ser sustituible. Elige la relación por el modelo del dominio, no por ahorrar una declaración. Si PrecioFinal conociera las campañas concretas, cada cambio obligaría a modificarlo y recompilar sus ramas. La interfaz permite que el mismo contrato use varias políticas sin acoplar el cálculo al catálogo de promociones.

Práctica: ¿Qué ventaja tiene inyectar una política de descuento frente a crear una subclase por campaña?

> Respuesta razonada: La política se puede sustituir en pruebas o en ejecución sin multiplicar subclases. Herencia encaja cuando el subtipo puede reemplazar a la base sin cambiar la expectativa del llamador.

Documentación consultada: Dev.java: programación orientada a objetos

# Interfaces y despacho dinámico

Objetivo: Procesar avisos de distintos tipos mediante un contrato común.

Una interfaz declara capacidades compartidas sin exigir que las clases hereden de una misma base. Al llamar a texto() mediante Aviso, Java despacha en ejecución la implementación del objeto real. Una clase abstracta puede compartir estado o código, pero no hace falta si solo se comparte un contrato. El consumidor conserva una referencia del tipo Aviso y evita decidir por nombre de clase qué método llamar.

Código · Java

```
interface Aviso { String texto(); }
record Nota(String texto) implements Aviso {}
record ErrorCarga(String texto) implements Aviso {}
class PresentadorAvisos {
  static void mostrar(Aviso aviso) {
    System.out.println(aviso.texto());
  }
}
```

El método mostrar recibe Aviso, no una clase particular; la llamada aviso.texto() se resuelve según el objeto real. Esa selección dinámica evita cadenas de if por cada variante y permite añadir una implementación sin tocar el cliente. Una interfaz debe expresar solo operaciones que todas sus implementaciones puedan cumplir. Si una variante no puede producir texto, revisa el contrato en vez de esconder un caso especial en el consumidor.

Práctica: Añade AvisoVacio y pásalo a mostrar. ¿Qué implementación necesita y qué consumidor debe cambiar?

> Respuesta razonada: AvisoVacio implementa texto() y devuelve una representación válida, quizá una cadena vacía o un mensaje acordado. Si cumple el contrato, mostrar no cambia. La clase nueva añade comportamiento sin una rama por tipo; el diseño sigue siendo extensible y comprobable mediante la interfaz común.

Documentación consultada: Java Language Specification, Java SE 21 · Dev.java: programación orientada a objetos

# Colecciones y garantías de orden

Objetivo: Elegir entre List, Set y Map según orden, duplicados y búsquedas por clave.

List conserva posiciones y repetidos; Set modela pertenencia sin duplicados; Map asocia claves únicas a valores. HashMap no promete orden de iteración, LinkedHashMap conserva orden de inserción y TreeMap ordena por clave. Al diseñar una colección de claves propias, define igualdad y hashCode de forma coherente. Prueba la garantía que necesita el informe, no un orden incidental observado en una ejecución.

Código · Java

```
Map<String, Integer> stock = new LinkedHashMap<>();
stock.merge("cable", 1, Integer::sum);
stock.merge("cable", 1, Integer::sum);
System.out.println(stock.get("cable")); // 2
```

LinkedHashMap conserva el orden de inserción; TreeMap ordena por clave. HashMap no garantiza ninguno de esos órdenes. merge actualiza el valor asociado y crea una cuenta cuando la clave aún no existe. Si las claves son objetos, igualdad y hashCode gobiernan su búsqueda: claves iguales deben compartir hash. Para un informe alfabético, TreeMap evita ordenar una lista a mano y hace explícito el requisito.

Práctica: Cuenta artículos repetidos con un Map y presenta las claves alfabéticamente. Si usas objetos Producto como claves, ¿qué contrato deben respetar equals y hashCode?

> Respuesta razonada: Un Map acumula frecuencias; TreeMap ordena las claves para presentar el informe. HashMap sirve para contar si no se exige orden. Si dos Producto se consideran iguales, equals debe ser simétrico y coherente, y ambos deben producir el mismo hashCode; de lo contrario un Set o Map puede no encontrar una clave que ya contiene.

Documentación consultada: API Java SE 21: colecciones

# Excepciones y ficheros

Objetivo: Leer un fichero UTF-8 y comunicar un error sin fingir una lectura correcta.

Path representa una ruta y Files ofrece operaciones de lectura. Files.newBufferedReader permite recorrer texto en UTF-8 sin cargar el fichero completo; try-with-resources cierra el lector aunque falle la lectura. Captura IOException solo donde puedas aportar contexto o una recuperación útil. Para archivos pequeños, readAllLines es cómodo, pero también consume memoria proporcional al contenido.

Código · Java

```
Path ruta = Path.of("notas.txt");
try (BufferedReader lector = Files.newBufferedReader(
        ruta, StandardCharsets.UTF_8)) {
  String linea;
  while ((linea = lector.readLine()) != null) {
    System.out.println(linea);
  }
} catch (IOException ex) {
  System.err.println("No se pudo leer: " + ex.getMessage());
}
```

Files.newBufferedReader permite procesar el fichero línea a línea sin cargarlo entero. try-with-resources cierra el lector incluso si readLine o la salida falla. Fijar UTF-8 evita depender de la codificación por defecto. IOException debe comunicarse con contexto útil; no se debe sustituir por una lista vacía que parezca una lectura correcta. Para archivos pequeños, readAllLines es cómodo, pero también consume memoria proporcional al contenido.

Práctica: El fichero no existe. ¿Qué información debe recibir el usuario y qué detalle registrarías para distinguirlo de un error de codificación o permisos?

> Respuesta razonada: Informa que la lectura ha fallado y conserva la causa para depuración; no devuelvas contenido inventado ni una lista vacía que parezca correcta. UTF-8 evita depender de la codificación por defecto. Si el usuario puede reparar una ruta o permiso, indica el siguiente paso sin exponer una traza interna innecesaria.

Documentación consultada: API Java SE 21: Files

## Fuentes oficiales

Java Language Specification, Java SE 21 · Dev.java: programación orientada a objetos · API Java SE 21: colecciones · API Java SE 21: Files · JUnit 5.14.4 · User Guide · API Java SE 21: String.equals